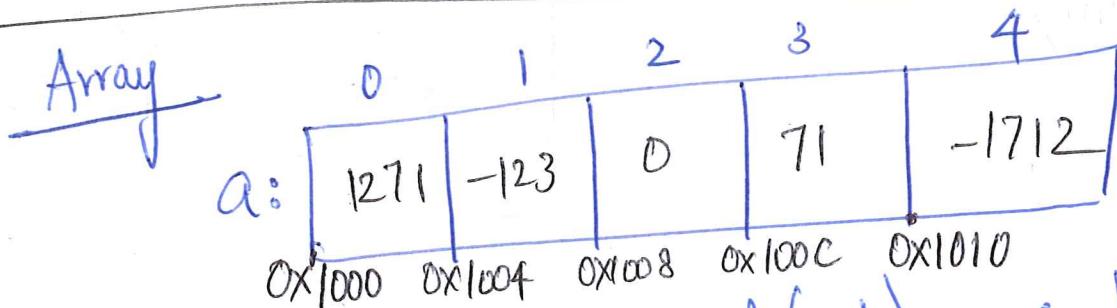


CS 354 - Lecture 3

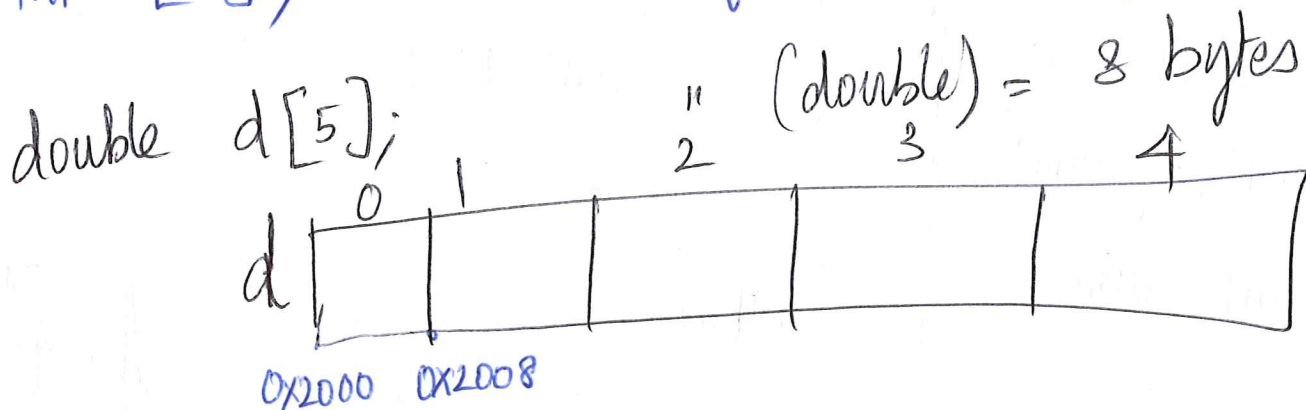
1. Worksheets - 15% ①
2. project 2

Today

1. Review
2. Dynamic Memory Allocation
3. Linked Lists ✕
4. File I/O



int a[5]; sizeof(int) = 4 bytes

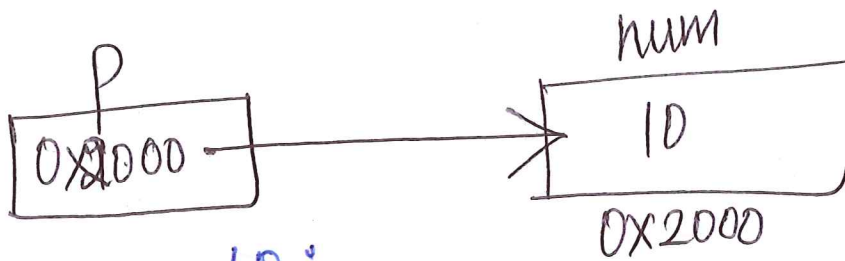


a[0] - a[4]

a[5], a[6]

Pointers

(2)



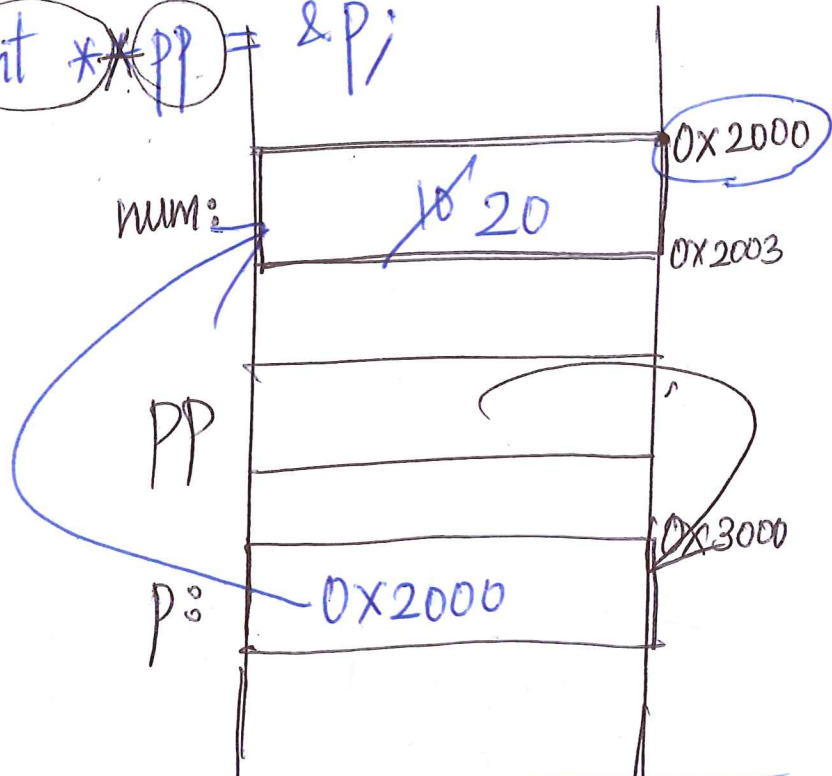
`int num = 10;`

`int * p;`
←
`int * p;`
`int *p;`

`p = #`

`*p = 20;`

`int **pp = &p;`



```
main() {  
    int num = 10;  
    addOne(num);  
    print num;  
}
```



```
void addOne(int num) {  
    num = num + 1;  
}
```

addOne:

num: 10

main

num: 10 0x1000

stack frame for addOne

SF for main

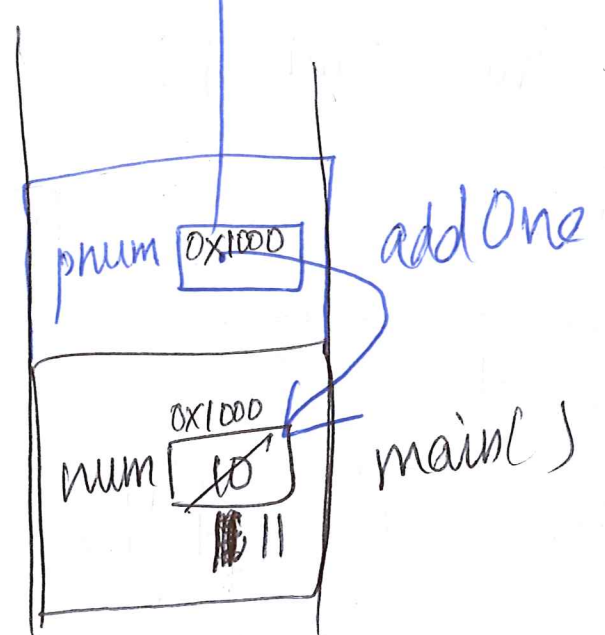
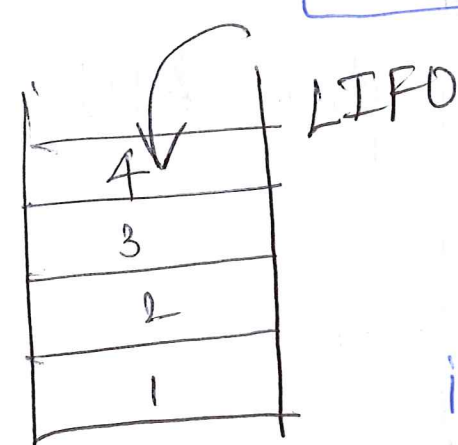
(3)

```
void addOne (int *pnum) {  
    (*pnum) = *pnum + 1;  
}
```

In main()
addOne(&num);

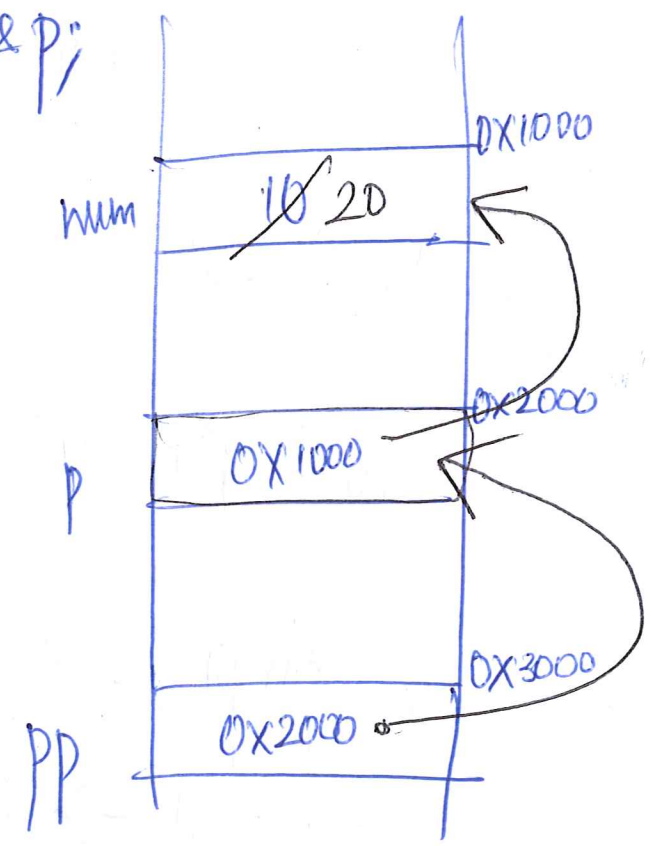
n = n + 1;
10

n = 11



int **pp = &p;

**pp = 20;



structures

④
(x, y)

struct point {
int x;
int y;
};

int num;

struct point p1;

p1.x = 10;

p1.y = 7;

p1

x:	10	20
y:	7	3

0x3000

ptrpt = &p1;

struct point *ptrpt;
= NULL;

ptrpt

0x1230

0x4000

(*ptrpt).x

struct point *ptrpt = &p1;

(*ptrpt).x = 20;

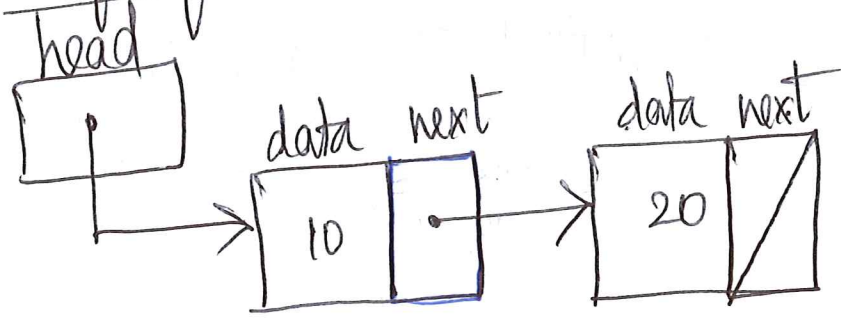
ptrpt -> y = 3;

Arrays of structures ⁽⁵⁾

struct point pt[5];

pt[i].x = value;

Self referential structures



struct node {
int data;

→ 4 bytes

struct node * next;

→ 4 bytes

};

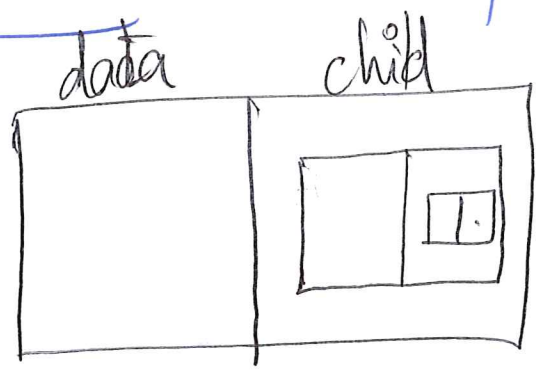
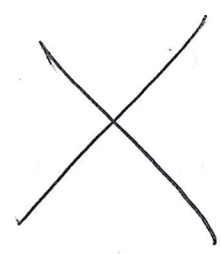
	x	y
p[0]		
1		
2		
3		
4		

p[0]	x	y
p[1]	x	y

struct node {
int data;

struct node child;

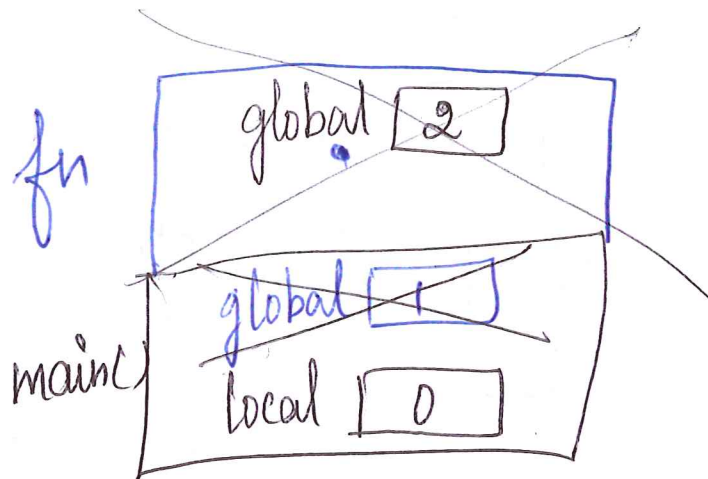
}



```

int global = 1; ⑥
main ( ) {
    → int local = 0;
    → fn(...);
}

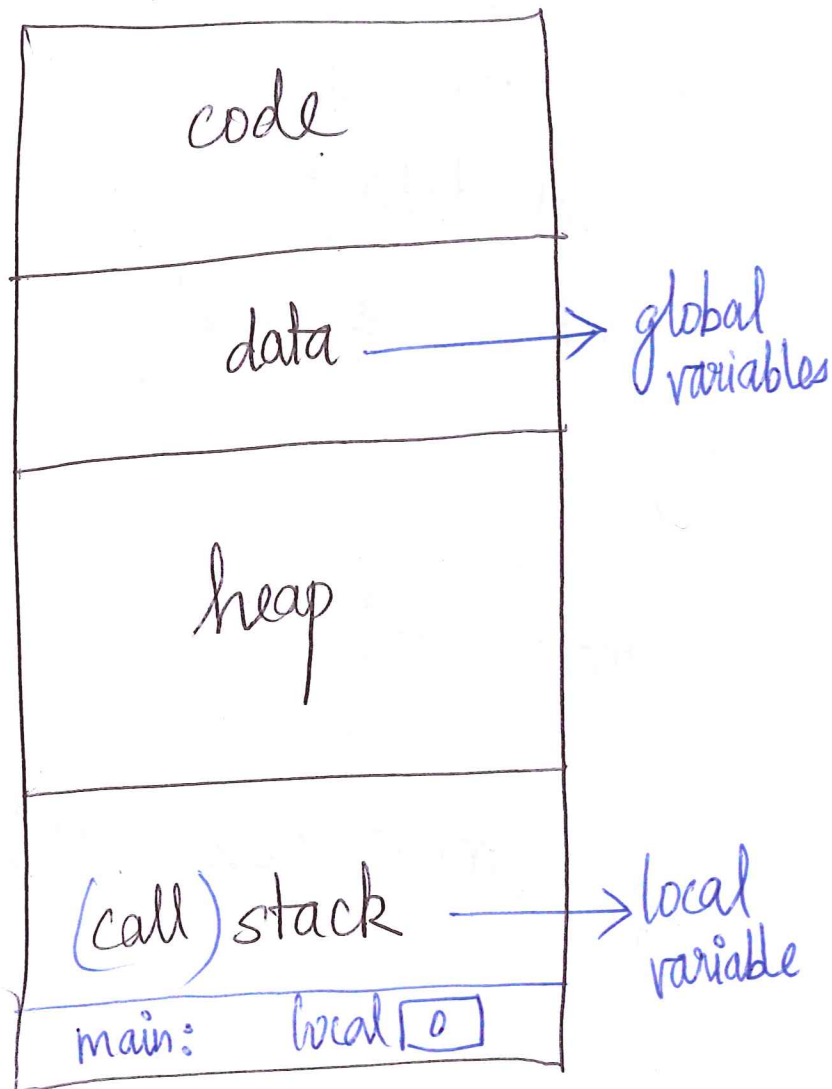
```



```

→ fn ( ) {
    global = 2;
}

```



Dynamic Memory Allocation. ⁽⁷⁾

get request memory from heap.

void * malloc (size_t size);
↓
unsigned integer.

void * p = malloc (8);

int * p = malloc (8);

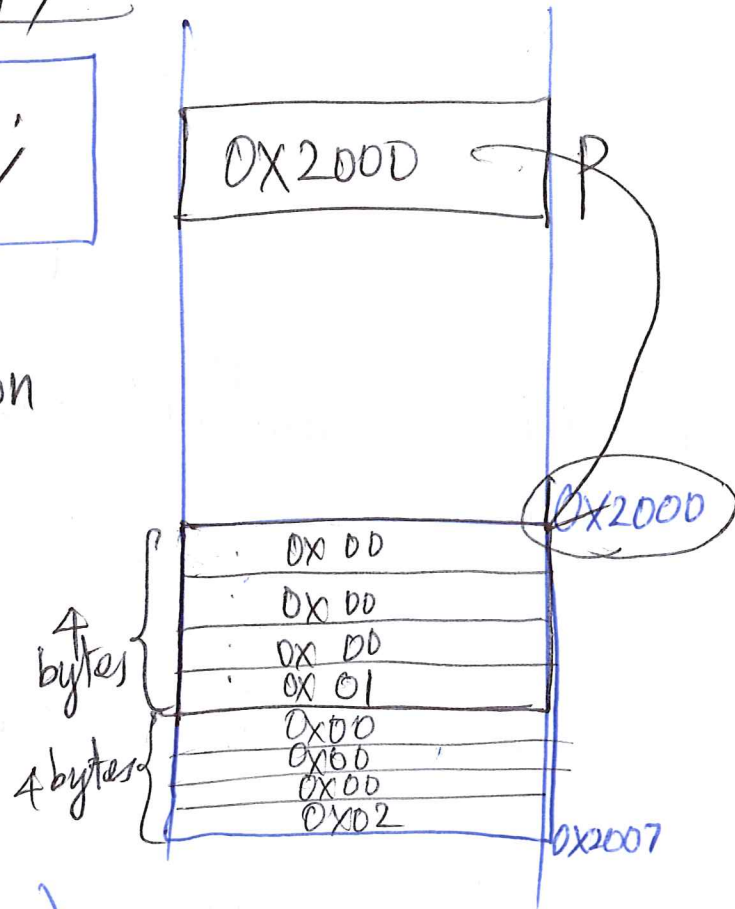
Contiguous region of memory

p[0] = 1;

*(p+i) = 1;

*(p+0) = 1;

*p = 1;



*(p+1) = 2;

0x2000 + 1 * 4

0x2004 sizeof(int)

int

4 bytes

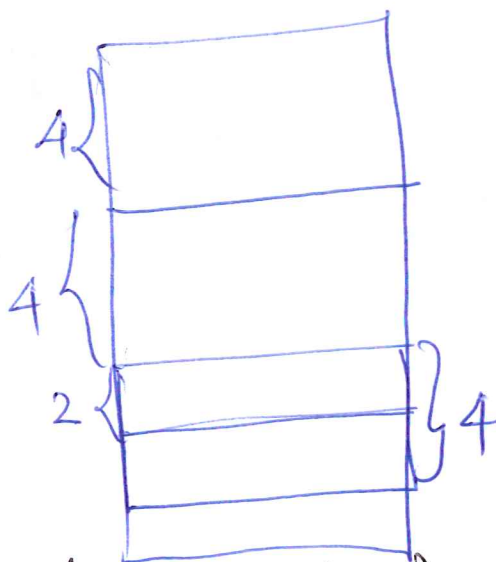
⑧

32 bits

32
2 - 1

malloc(10)

0 0 0 0
1 1 1 1
+ 1
0 0 0 0



int *p = malloc(sizeof(int) * 10);

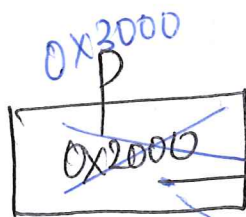
p = malloc(sizeof(int) * 1000);

NULL → malloc failed.
No more memory in heap!

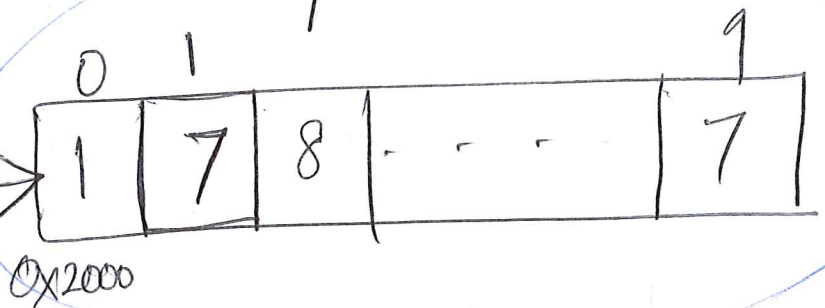
if (p == NULL) {
}

Need more memory? ⑨

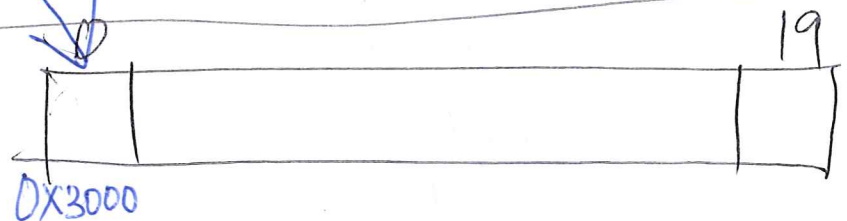
Stack



Heap

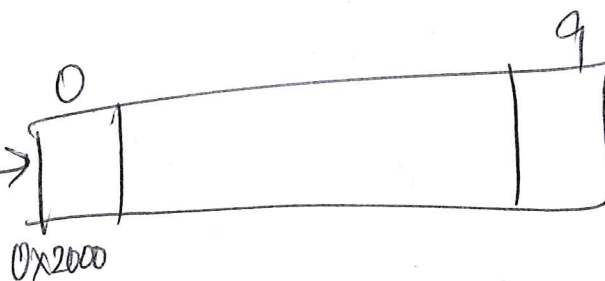
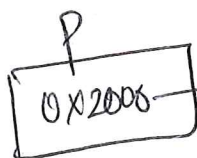


`p[1] = 7;`



`p = malloc(sizeof(int) * 20);`

`free(p);`

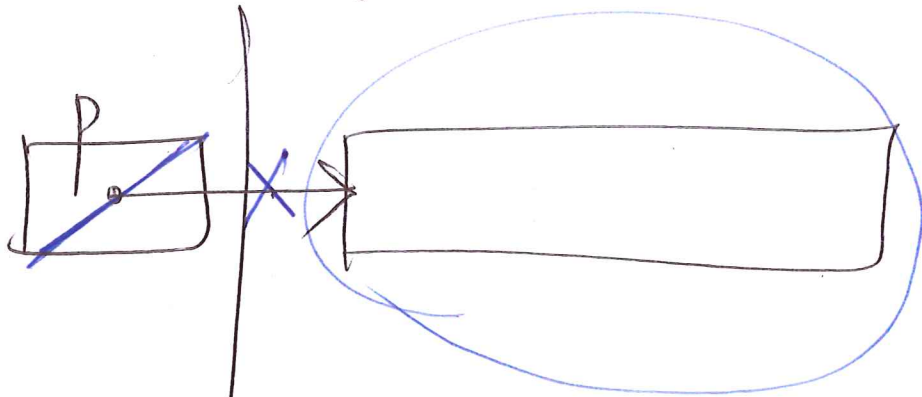


$\text{int } *p = \text{malloc}(\overset{(10)}{8});$

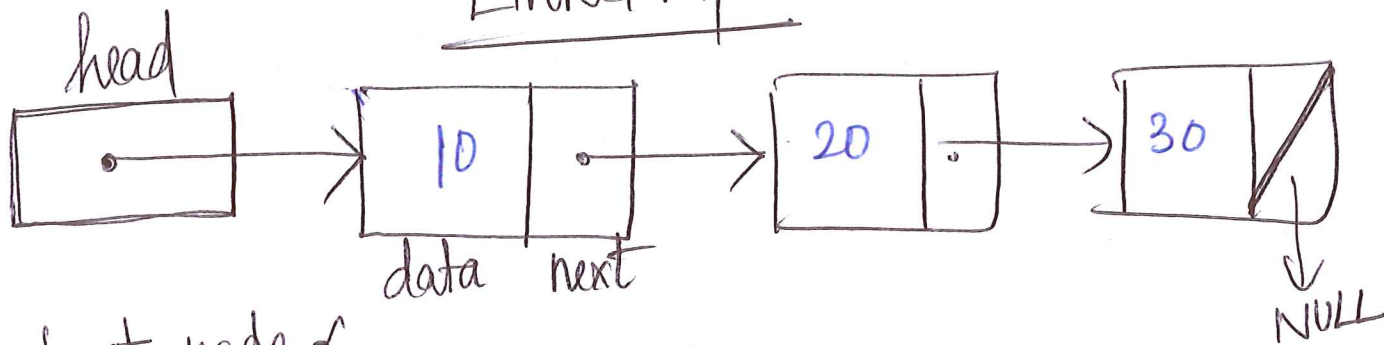
$p = \text{NULL};$ // memory leak!

$\text{free}(p);$

$\downarrow$
 $\text{free}(\text{NULL});$
// no op.



Linked List



```
struct node {  
    int data;  
    struct node * next;  
};
```

main () { ^⑪

① struct node *head = NULL;

② ~~struct~~ head = malloc (sizeof (struct node));

head → data = 10;

head → next = NULL;

}

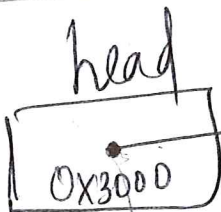
stack

heap

①

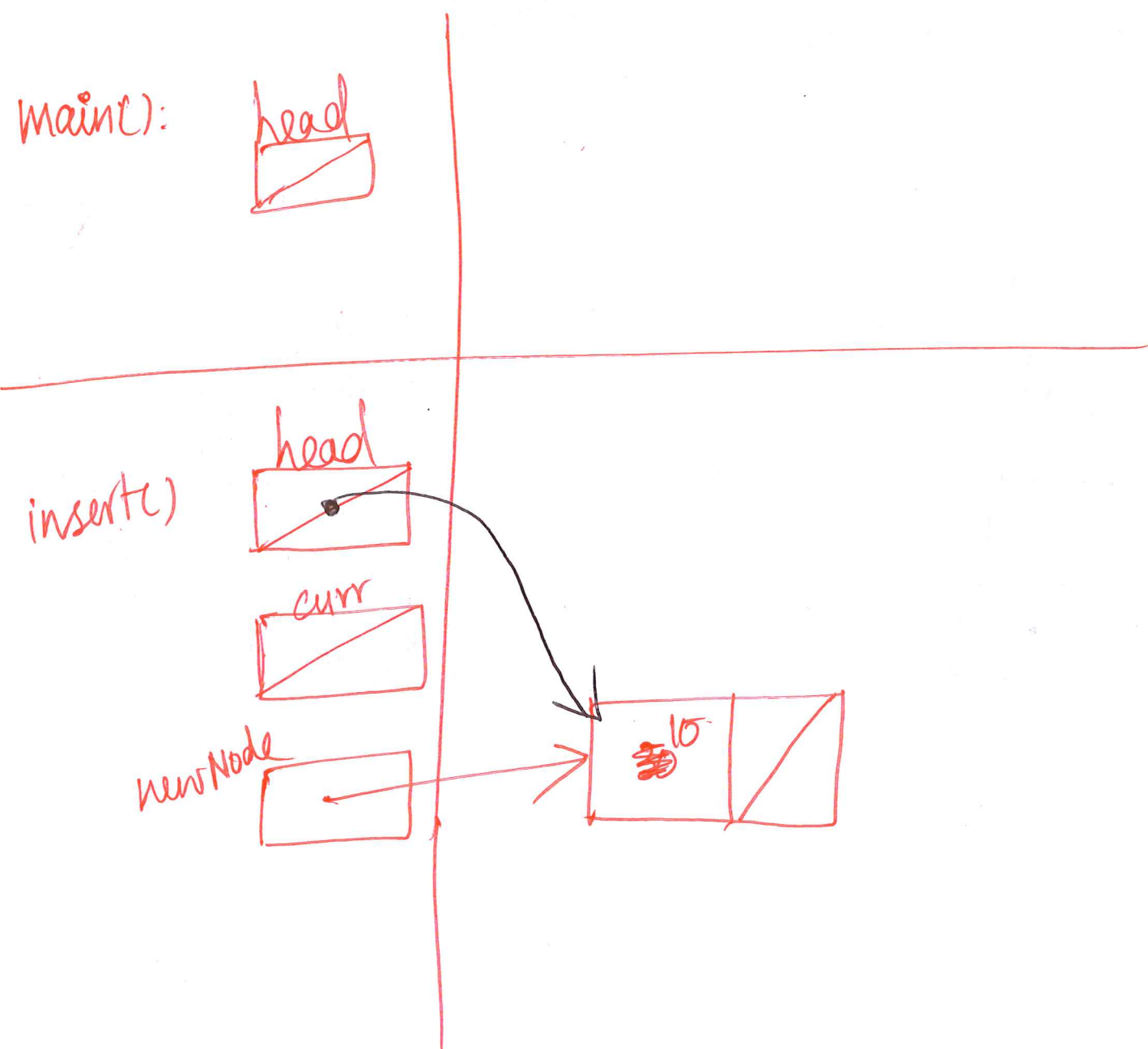
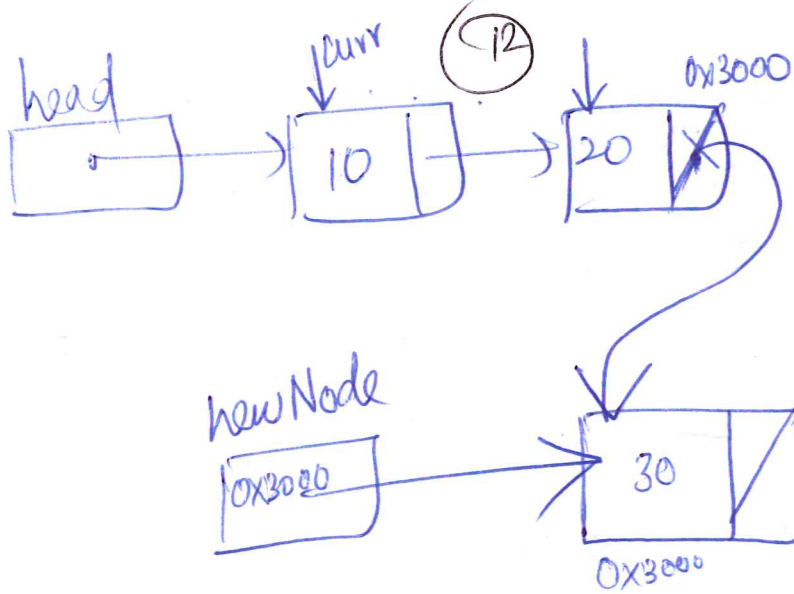


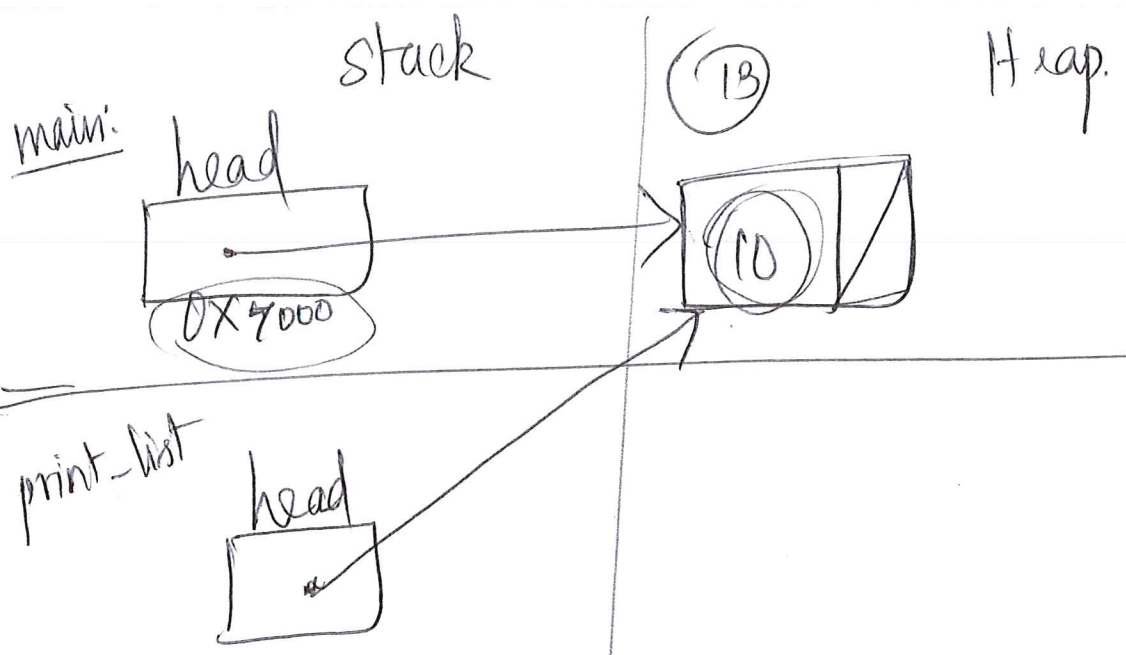
②



0x3000







```

int delete (struct node * * phead) {
    assert(first != NULL);
    struct node * first = *phead;
    int data = first->data;
    *phead = first->next;
    free(first);
    return data;
}

```

